Cisco Confidential

ACTIVIDAD PRACTICA

Diseño arquitectural del sistema con IA

Guía de trabajo para alumnos

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto.

Empresa asignada	PetMarket
Integrantes	Emiliano Sánchez Juárez2169669 Valeria Nohemí Cisneros Moreno2178081
Fecha de entrega	10/12/2026

Cisco Confidential

6 Etapa 1 Preparar el caso

Seleccionen de tres a cinco casos de uso relacionados con el proceso principal. Deben incluir al menos un flujo normal y un flujo con excepción o rechazo. Para cada caso de uso, identifiquen el actor, el objetivo, la información de entrada, las reglas involucradas y el resultado esperado.

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Iniciar checkout y validar inventario	- Cliente (principal) - PetMarket Plataforma (secundario)	**RN-01:** El stock debe estar disponible en el almacén. **RN-02:** En caso que no haya stock, notificar “producto agotado” para que el cliente ajuste su carrito de compras. **RN-03:** Solo se avanza a la validación de dirección y método de entrega si el stock es suficiente.	*Normal:* Inventario confirmado y checkout exitoso. *Excepción:* El cliente es notificado del producto agotado y ajusta su carrito
CU-02	Procesar pago y confirmar pedido	- Cliente (principal) - Pagos (secundario)	**RN-04:** La dirección y el método de entrega deben confirmarse válidos antes del pago. **RN-05:** El pedido se confirma solo si el pago es aprobado. **RN-06:** Si el pago es rechazado, se notificará al cliente y se volverá a intentar.	*Normal:* Pago aprobado y pedido confirmado. *Excepción:* El cliente es notificado del pago rechazado y se vuelve a intentar.
CU-03	Preparar, empacar y despachar pedido	- Almacen (principal) - Paqueteria/Reparto (secundario)	**RN-07:** Si el pedido requiere 2 almacenes, cada uno prepara su línea. **RN-08:** Si requiere un solo almacén, este prepara el pedido completo. **RN-09:** El pedido se empaca y consolida antes del despacho**.** **RN-10:** Si la entrega es exitosa, el proceso finaliza como “pedido entregado”. **RN-11:** Si la entrega no es exitosa, se identificará el área de oportunidad y se empaquetara un pedido nuevo al cliente, notificándose del cambio.	*Normal:* Pedido preparado, consolidado, despachado y entregado al cliente. *Excepción:* La entrega del pedido se retrasa y es entregado al cliente en una fecha diferente.

Cisco Confidential

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-04	Cancelar pedido antes del envío	- Cliente (principal) - Atención al Cliente (secundario)	**RN-12:** La cancelación es posible en cualquier etapa, mientras sea antes del envío. **RN-13:** Al cancelar se registra la cancelación y se notifica al almacén. **RN-14:** La cancelación genera un reembolso por cancelación.	*Normal:* Cancelación registrada, almacén notificado, reembolso emitido. *Excepción:*

Antes de continuar, comprueben que los casos seleccionados aparecen en las historias de usuario o en el BPMN. Si no existe esa relación, el caso debe justificarse como corrección de alcance.

7 Etapa 2 Elegir el patron de logica de negocio

Comparen Transaction Script, Table Module y Domain Model. La decisión deberá basarse en el comportamiento del sistema: cantidad de reglas, interacción entre conceptos, cambios esperados, manejo de estados y necesidad de proteger invariantes.

Trabajo solicitado

- Construyan una tabla comparativa con criterios propios del caso.
- Incluyan ventajas, riesgos y consecuencias de cada alternativa en su empresa.
- Seleccionen un patrón principal. Pueden proponer una combinación solo si delimitan dónde se usa cada patrón.
- Relacionen la decisión con al menos tres evidencias de la Unidad 2.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Cada caso de uso es un procedimiento con sus if. Con 13 reglas (stock, pago, 1 o 2 almacenes, cancelación) se duplican condiciones entre scripts.	Agrupa las reglas por tabla (Pedidos, Pagos, Inventario), pero las reglas que cruzan tablas (pago, stock y envío) quedan sin dueño claro.	Cada regla vive en el objeto al que pertenece (Pedido, Pago, Envío), sin repetirse.	Reglas de negocio derivadas del BPMN (reglas previamente establecidas, RN-01 a RN-14)
Cambios esperados	Cambiar una regla, como agregar un tercer almacén, obliga a revisar muchos scripts. Riesgo alto de olvidar alguno.	Los cambios se localizan por tabla, pero siguen afectando varios módulos.	El cambio se concentra en una clase o política (p. ej. la asignación de almacén). Es el que mejor lo absorbe.	Glosario U2: Pedido dividido, Almacén único y Método de entrega, variantes que cambian con el negocio

Cisco Confidential

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Interacción entre conceptos	Los conceptos son solo datos que el script manipula. Las relaciones Pedido, Línea, Almacén y Pago se codifican a mano.	Cada módulo trabaja su tabla y la coordinación entre módulos recae en código externo.	Las relaciones (un Pedido tiene Líneas, una Línea se surte desde un Almacén, un Pedido tiene Pagos y Envíos) se modelan directamente.	Glosario U2: Pedido dividido, Almacén, Pago, Envío y Reembolso
Estados e invariantes	El estado es un campo que cualquier script puede modificar. Nada impide un pedido “despachado” sin pago aprobado.	Se valida a nivel de registro, pero no se protege el ciclo de vida completo.	El Pedido controla sus transiciones (confirmado, preparado, despachado, entregado o cancelado) y rechaza las inválidas.	BPMN U2: gateways ¿Pago aprobado?, ¿Pedido requiere 2 almacenes?, ¿Cliente cancela antes del envío?
Criterio adicional	Mínimo costo inicial. Ideal para operaciones simples y aisladas.	Costo bajo a medio. Encaja si el sistema gira en torno a tablas y reportes.	Costo inicial más alto (diseño, mapeo a base de datos, curva de aprendizaje).	Alcance del BPMN: un proceso principal con dos almacenes

Ventajas, riesgos y consecuencias en PetMarket

Patrón	Ventajas	Riesgos	Consecuencias en la empresa
Transcription Script	Rápido de construir, fácil de entender, poco diseño previo.	Lógica duplicada, scripts largos, invariantes sin protección.	Se lanza pronto, pero cada nueva regla (promociones, nuevo almacén) encarece el mantenimiento y sube la probabilidad de errores en pedidos y reembolsos.
Table Module	Orden por tabla, buen apoyo de herramientas y reportes.	Las reglas que cruzan módulos quedan dispersas. No modela bien el flujo de un pedido en varios almacenes.	Funciona para catálogos e inventario, pero el proceso de pedido seguiría fragmentado.
Domain Model	Reglas e invariantes centralizadas, extensible, refleja el lenguaje del negocio y el BPMN.	Más tiempo inicial, requiere disciplina de diseño, riesgo de sobrediseño en partes simples.	Mayor inversión al inicio, pero menos errores costosos (envíos sin pago, cancelaciones tardías) y

Cisco Confidential

			cambios más baratos a mediano plazo.

Cisco Confidential

En base a la información ya presentada, el patrón principal a usar será el Domain Model, el cual se aplica al núcleo del proceso del pedido (Pedido, Línea de pedido, Pago, Envío, Reembolso y Almacén con su inventario). Si habrá una combinación delimitada con Transaction Script, pero solo en operaciones simples y sin reglas cruzadas, como notificaciones para el cliente o para generar consultas y reportes de solo lectura.

Los problemas que resuelve dentro del caso son las invariantes del pedido, interacción entre varios conceptos y los cambios esperados:

- Invariantes del pedido: Las condiciones que se imponen y no deben cambiar son el que el pedido no se confirma hasta aprobarse el pago (RN-05), no se despacha sin empacar y consolidar (RN-09) y no se cancela después del envío (RN-11). En este caso, el patrón más adecuado sería el Domain Model ya que controla sus propias transiciones y rechaza las inválidas.
- Interacción entre varios conceptos. El caso del pedido dividido (RN-07 y RN-08) obliga a que un mismo Pedido reparta líneas entre almacenes y luego las consolide. Es una relación entre objetos, no un procedimiento lineal, y se expresa de forma natural con un modelo de objetos.
- Cambios esperados. Una tienda en crecimiento probablemente añadirá almacenes, métodos de pago y políticas de devolución. Con Domain Model, esos cambios se concentran en pocas clases en lugar de repartirse por toda la aplicación.

En cuanto al costo, el equipo debe invertir más tiempo al inicio en diseñar el modelo y mapearlo a la base de datos, y necesita conocer el patrón. Para controlarlo, se aplica sólo al núcleo del pedido y no a todo el sistema. Si el proyecto tuviera un plazo muy corto y reglas estables, Transaction Script sería más razonable, pero este no es el caso.

Entre su relación con las evidencias de la Unidad 2, se encuentran las reglas de negocio derivadas del BPMN, ya que su cantidad y el cruce entre ellas justifican centralizarlas en el dominio. También está el BPMN en sí, con los gateways (¿pago aprobado?, ¿pedido requiere 2 almacenes?, ¿cliente cancela antes del envío?) y las ramas de excepción muestran estados y transiciones que el Pedido debe proteger. Y por último, el glosario con términos como Pedido dividido, Almacén, Pago y Reembolso son la base de las clases del modelo.

Producto de la etapa: una decisión argumentada de una a dos páginas. No basta escribir que un patrón es más fácil, moderno o escalable. La explicación debe indicar qué problema concreto del caso ayuda a resolver y qué costo introduce.

8 Etapa 3 Construir el mapa de dominio

Transformen el lenguaje y las reglas de la Unidad 2 en un mapa de responsabilidades. Identifiquen los elementos que el sistema debe reconocer, proteger o comunicar. No conviertan automáticamente cada tabla de datos en una entidad del dominio.

Elementos que deben identificar

Elemento	Pregunta que debe responder el equipo
Lenguaje ubicuo	Terminos usados por usuarios, analistas y desarrolladores con el mismo significado.

Cisco Confidential

Elemento	Pregunta que debe responder el equipo
Entidades	Conceptos con identidad y ciclo de vida que deben distinguirse aun cuando cambien sus datos.
Value Objects	Conceptos definidos por sus valores y reglas, sin identidad propia relevante.
Aggregates	Grupos que requieren un limite de consistencia y una raiz para controlar cambios.
Domain Services	Comportamientos del dominio que no pertenecen naturalmente a una sola entidad.
Domain Events	Hechos del negocio que ya ocurrieron y pueden interesar a otros componentes.

Producto de la etapa

- Diagrama del mapa de dominio con relaciones y límites claramente identificados.
- Glosario actualizado con los términos nuevos o corregidos.
- Ficha de cada Aggregate que indique su raíz, responsabilidades, invariantes y eventos posibles.
- Explicación de las decisiones dudosas o descartadas.

Glosario actualizado

Término	Rol
Pedido dividido (split order)	Variante del Pedido cuyas líneas salen de dos almacenes. Es un estado o tipo del Pedido, no un aggregate
Almacen unico/Preparar pedido	Variante del Pedido que sale completo de un solo Almacén
Almacén	Raíz del Aggregate Almacén (dos instancias: Almacén 1 y 2).
Inventario	Conjunto de Existencias dentro de cada Almacén.
Preparar línea	Comportamiento del Almacén. Genera el evento LíneaPreparada.
Empacar y consolidar pedido	Comportamiento del Pedido. Genera PedidoConsolidado.
Confirmar pedido	Comportamiento del Pedido. Solo es válido con pago aprobado. Genera PedidoConfirmado.
Procesar pago / Pago rechazado	Comportamiento y resultado del Pago. Generan PagoAprobado o PagoRechazado.
Producto agotado	Resultado de validar el inventario. Genera ProductoAgotado.
Entrega fallida / Entrega exitosa	Resultado del Envío. Generan EntregaFallida o PedidoEntregado.
Cancelación	Comportamiento del Pedido. Genera PedidoCancelado.

Cisco Confidential

Reembolso por cancelación / por devolución	Los dos tipos del Aggregate Reembolso.
Método de entrega	Value Object del Pedido.
Cliente, Atención al Cliente, Paquetería / Reparto, PetMarket (Plataforma)	Actores y roles. No son Aggregates.
Stock disponible en almacén	Disponibilidad del Producto dentro del Almacen.
Carrito	Selección de productos del cliente antes de iniciar el checkout. Aún no es Pedido.
Pedido	Compra del cliente desde que inicia el checkout (estado “Pendiente de pago”) hasta que se entrega o cancela.
Línea de pedido	Producto y cantidad dentro de un Pedido, asignada a un almacén.
Existencia	Cantidad de un producto en un Almacén concreto
Pago	Intento de cobro de un Pedido, con su resultado
Pasarela de pagos	Servicio externo que autoriza la transacción (se menciona en el glosario, pero no se define).
Envío	Salida física del pedido hacia el cliente, con su resultado de entrega.
Reembolso	Devolución de dinero, con su tipo y motivo.
Reserva	Cantidad apartada de una Existencia para un Pedido, aún no despachada.

Fichas de Aggregates

	Raíz	Responsabilidades	Invariantes	Eventos
Pedido	**Raíz:** Pedido. **Contiene:** Líneas de pedido, Dirección, Método de entrega y Dinero (total).	Confirmar el pedido, repartir líneas por almacén (único o dividido), empacar y consolidar, y cancelarse.	• No se confirma sin pago aprobado. • No se despacha sin empacar y consolidar. • Un Pedido dividido no se consolida hasta que ambas líneas estén preparadas. • No se cancela después del despacho. • El total es la suma de sus líneas.	PedidoConfirmado, PedidoConsolidado, PedidoDespachado, PedidoEntregado, PedidoCancelado

Cisco Confidential

Almacén	**Raíz:** Almacén. **Contiene:** Existencias con su Cantidad.	Validar disponibilidad, reservar y preparar líneas.	• La cantidad disponible nunca es negativa. • No se prepara una línea con más cantidad de la que existe.	ProductoAgotado, LíneaPreparada
Pago	**Raíz:** Pago. **Contiene**: Dinero, Método de pago y resultado.	Enviar la transacción a la pasarela, registrar aprobación o rechazo y permitir reintentos.	• El monto coincide con el total del Pedido. • Un pago aprobado no se procesa dos veces. • Solo se reintenta un pago rechazado.	PagoAprobado, PagoRechazado
Envío	**Raíz**: Envío. **Contiene:** Dirección destino y resultado de entrega.	Despachar y registrar entrega exitosa o fallida.	• Solo existe para un Pedido ya consolidado. • No pasa a “entregado” sin haber sido despachado. • Una entrega fallida deriva en devolución o gestión de Atención al Cliente.	PedidoDespachado, PedidoEntregado, EntregaFallida
Reembolso	Raíz: Reembolso. Contiene: Dinero, Motivo y Tipo (por cancelación o por devolución).	Devolver el dinero al cliente.	• Solo se reembolsa un pago aprobado. • El monto no excede lo pagado. • Un pago no se reembolsa dos veces.	ReembolsoEmitido

Decisiones descartadas

- **Pago dentro del Aggregate Pedido.** Se descartó porque el pago tiene su propio ciclo de vida: puede aprobarse, rechazarse y reintentarse varias veces, y lo ejecuta otro actor (el carril de Pagos en el BPMN). Si el Pago viviera dentro del Pedido, cada intento de cobro bloquearía todo el pedido, aunque el cobro no tenga nada que ver con las líneas ni con el envío. Por eso Pago es un Aggregate separado que se relaciona con el Pedido por su identificador. Así el Pago protege sus propias reglas (monto igual al total, un pago aprobado no se repite) sin interferir con el Pedido.

- **Inventario dentro del Pedido.** Se descartó porque muchos pedidos compiten por las mismas existencias al mismo tiempo. Si el inventario fuera parte del Pedido, el límite de consistencia sería enorme y habría bloqueos y cuellos de botella cada vez que dos clientes compren el mismo producto. La disponibilidad pertenece al Almacén, que es quien protege la regla de que la cantidad nunca puede ser negativa, y el Pedido solo guarda a qué almacén pertenece cada línea.

- **Almacén 1 y Almacén 2 como Aggregates distintos.** Se descartó porque ambos almacenes siguen exactamente las mismas reglas. Lo único que cambia entre ellos es su identidad, no su comportamiento, así que se modelan como un solo Aggregate Almacén con dos instancias. Crear

Cisco Confidential

dos clases habría duplicado reglas y obligado a modificar ambas ante cualquier cambio. Esta decisión también facilita agregar un tercer almacén, algo que se anticipó en la Etapa 2.

- **Pedido dividido como Aggregate propio.** Se descartó porque dividir un pedido solo cambia cómo se reparten las líneas entre almacenes, no las reglas del pedido. Un pedido dividido sigue necesitando pago aprobado para confirmarse, sigue consolidándose antes del despacho y sigue sin poder cancelarse después de despachado. Por eso se maneja como un estado del Pedido (el atributo dividido). En la Etapa 7 se confirmó que modelarlo como subclase tampoco era buena idea, porque una subclase cambiaría la precondición de consolidar() y no podría sustituir al Pedido sin alterar su comportamiento.

- **Cliente como Aggregate.** Se descartó en esta etapa porque, en el BPMN, el cliente solo aparece como actor que inicia el proceso, decide el método de entrega y puede cancelar. El proceso no define reglas propias del cliente que haya que proteger, así que basta con referirse a él por su identificador (IdCliente). Si más adelante aparecen reglas propias (por ejemplo, cuentas, direcciones guardadas o historial), esta decisión se reconsidera.

- **Atención al Cliente y Paquetería como Aggregates.** Se descartaron porque son roles y actores del proceso, no conceptos con estado ni invariantes que proteger. Atención al Cliente dispara la cancelación o gestiona una entrega fallida, y Paquetería ejecuta la entrega, pero ninguno tiene reglas propias que el dominio deba proteger. Sus intervenciones se representan con servicios de aplicación y puertos, no con Aggregates.

- **Convertir cada tabla en una entidad.** No se aplicó, porque la actividad pide expresamente no hacerlo. Por ejemplo, “Dirección” y “Método de entrega” pueden ser tablas en la base de datos, pero en el dominio son Value Objects: se definen por sus valores y sus reglas de validez, y no tienen identidad propia que seguir en el tiempo. Tratarlos como entidades habría añadido ciclo de vida y complejidad sin ningún beneficio.

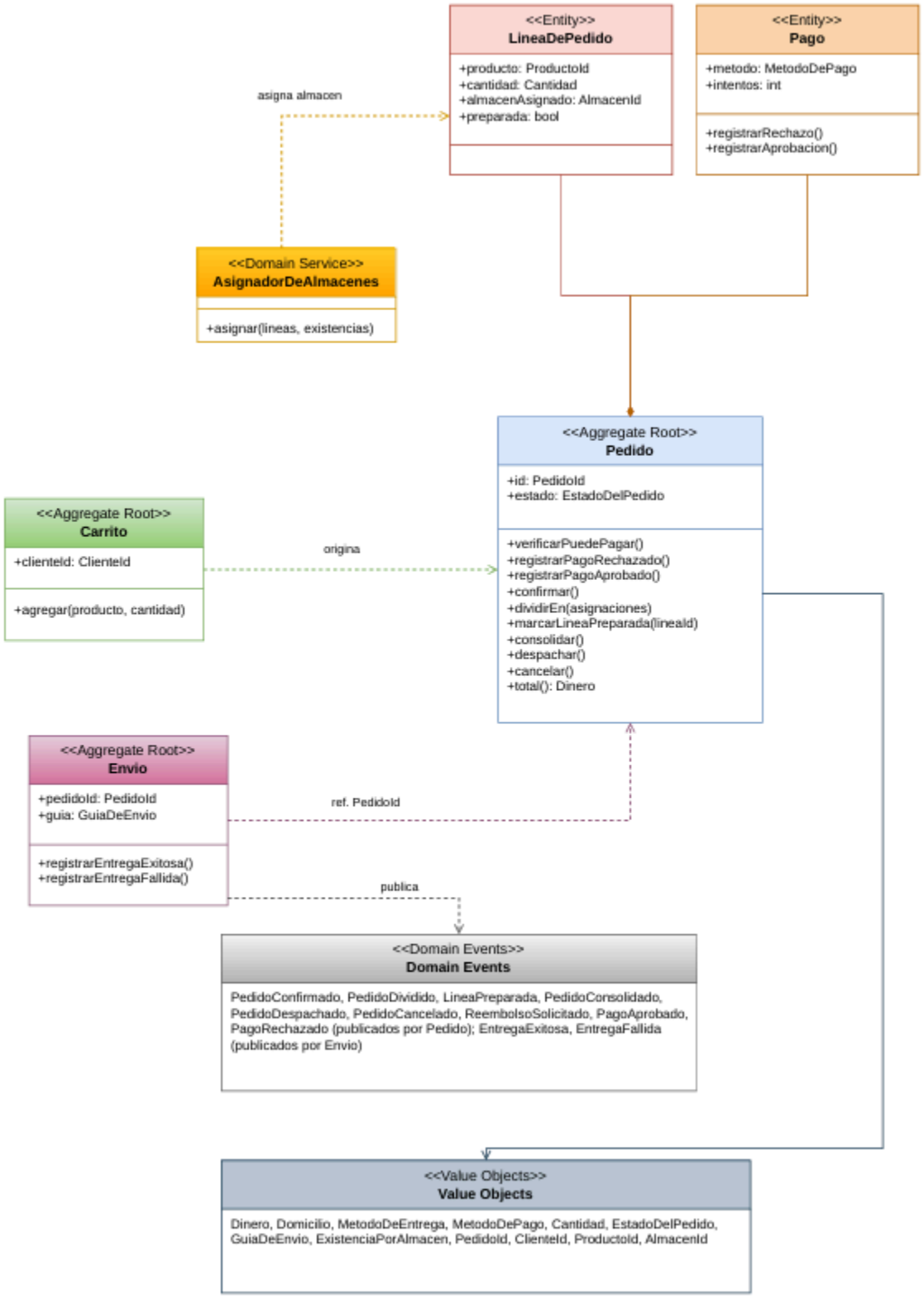
Decisiones dudosas

Además de las descartadas, tres decisiones se dejaron abiertas en lugar de descartarlas:

- **Carrito:** quedó fuera del mapa porque el BPMN no define reglas sobre él. Se reconsidera si aparecen carritos guardados o promociones.
- **Reserva de stock:** se modeló dentro de Existencia, sin Aggregate propio, porque el BPMN no muestra una reserva explícita. Se confirma si el equipo la necesita, pues implicaría liberar el stock al cancelar o rechazar el pago.
- **Devolución:** se absorbió en Envío y Reembolso porque el BPMN la muestra como una sola tarea. Se separaría si creciera, por ejemplo con inspección o reingreso a inventario.

Cisco Confidential

Mapa de Dominio



9 Etapa 4 Diseñar la Service Layer

Diseñen los Application Services que expondrán los casos de uso. Cada método debe representar una intención completa del usuario o de otro sistema. El servicio coordina la operación: recibe la solicitud, obtiene los objetos necesarios, invoca el comportamiento del dominio, solicita la persistencia y devuelve un resultado.

La Service Layer no debe convertirse en un lugar donde se acumulen todas las reglas. Si una condición describe lo que el negocio permite, calcula o protege, el equipo debe justificar dónde vive esa regla y por qué.

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
CheckoutService	`iniciarCheckout` (CU-01)	`IniciarCheckoutCommand` (carrito, domicilio, método de entrega)	`CarritoRepository`, `InventarioAlmacenRepository`, `PedidoRepository`, `UnitOfWork`	`ResultadoCheckout` (pedido creado o lista de productos agotados)	RN-01, RN-02, RN-03
PagoService	`procesarPago` (CU-02)	`ProcesarPagoCommand` (pedido, método de pago)	`PedidoRepository`, `PasarelaPago` (puerto), `UnitOfWork`	`ResultadoPago` (aprobado o rechazado con reintento disponible)	RN-04
PreparacionPedidoService	`dividirPedido`, `marcarLineaPreparada`, `consolidarPedido` (CU-03)	`PedidoId`, `AlmacenId`, `LineaId`	`PedidoRepository`, `InventarioAlmacenRepository`, `UnitOfWork`	Estado actualizado del pedido	RN-05, RN-06
CancelacionService	`cancelarPedido` (CU-04)	`CancelarPedidoCommand` (pedido, solicitante)	`PedidoRepository`, `PasarelaPago`, `UnitOfWork`	`ResultadoCancelacion` (cancelado con reembolso, o rechazada)	RN-07, RN-08

Cisco Confidential

Revision obligatoria

- ¿El nombre del método expresa una acción del negocio?
- ¿El método coordina o contiene decisiones que deberían estar en el dominio?
- ¿El servicio depende de interfaces o de detalles concretos de base de datos?
- ¿La operación tiene un inicio y un final transaccional comprensible?
- ¿Los errores del dominio pueden explicarse con términos del negocio?

10 Etapa 5 Separar persistencia y transacciones

Definan interfaces de Repository en términos del dominio y describan, por separado, cómo podrían implementarse en infraestructura. Los métodos deben expresar lo que el dominio necesita recuperar o guardar, sin exponer consultas SQL como parte del modelo.

También indiquen qué operaciones deben completarse juntas. La propuesta de Unit of Work debe explicar dónde inicia la transacción, qué cambios incluye y qué ocurriría ante un error.

Repository	Necesidad del dominio	Metodos propuestos	Objeto que recupera o guarda
PedidoRepository	Recuperar y guardar el pedido completo con sus líneas	`obtenerPorId(pedidoId)`, `guardar(pedido)`	Aggregate `Pedido`
CarritoRepository	Leer el carrito al iniciar checkout	obtenerPorCliente(clienteId)	Carrito
InventarioAlmacenRepository	Conocer la existencia por producto y almacén	`existenciaDe(producto)` → existencia por almacén	Valor de existencia (no se guarda desde el dominio)

11 Etapa 6 Separar Commands y Queries

Apliquen CQRS básico para hacer explícita la diferencia entre cambiar el estado y consultar información. No es obligatorio construir bases de datos separadas. El objetivo es que cada operación tenga una responsabilidad clara y que las consultas puedan devolver un modelo apropiado para lectura.

Trabajo solicitado

- Seleccionen al menos dos operaciones que cambien el estado y descríbanlas como Commands.
- Seleccionen al menos dos necesidades de información y descríbanlas como Queries.
- Definan el Handler de cada operación y sus dependencias.
- Diseñen el DTO o View Model de lectura sin reutilizarlo automáticamente como modelo de escritura.
- Justifiquen si alguna parte del caso no necesita esta separación.

Cisco Confidential

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	IniciarCheckoutCommand	carrito, domicilio, método de entrega	IniciarCheckoutHandler	`PedidoId` o lista de productos agotados	Cambia estado: crea el Pedido (CU-01)
Command	ProcesarPagoCommand	pedido, método de pago	ProcesarPagoHandler	aprobado, o rechazado con reintento disponible	Cambia el estado del Pago (CU-02)
Command	CancelarPedidoCommand	pedido, solicitante	CancelarPedidoHandler	cancelado con reembolso, o rechazado	Aplica RN-07 y RN-08 (CU-04)
Command	MarcarLineaPreparadaCommand	pedido, línea, almacén	MarcarLineaPreparadaHandler	estado actualizado	Cambia estado desde el almacén (US10)
Query	ConsultarEstadoDelPedido	PedidoId	ConsultarEstadoDelPedidoHandler	EstadoPedidoView	US13: el cliente consulta sin cambiar nada
Query	ConsultarCancelacionesYDevoluciones	rango de fechas	ReporteCancelacionesHandler	ReporteCancelacionesView	US18: reporte por periodo

12 Etapa 7 Revisar la propuesta con SOLID

Revisen la propuesta completa antes de cerrar los diagramas. El análisis debe mostrar decisiones concretas y no una definición memorizada de cada principio.

Principio	Pregunta de revision	Evidencia que deben buscar
S	¿Cada clase o servicio tiene una razon principal para cambiar?	Responsabilidades mezcladas, clases extensas, servicios que hacen de todo.
O	¿Pueden agregar una variante de regla sin editar una cadena creciente de condiciones?	Condicionales que deben modificarse por cada nueva politica.
L	¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado?	Subtipos que rechazan operaciones o alteran contratos.
I	¿Cada consumidor depende solo de las operaciones que utiliza?	Interfaces grandes con metodos irrelevantes para algunos clientes.
D	¿La aplicacion y el dominio dependen de abstracciones?	Dependencias directas de SQL, frameworks o servicios externos.

Cisco Confidential

Registren al menos cinco hallazgos. Un hallazgo puede confirmar una buena decisión o señalar una corrección necesaria. Para cada uno, incluyan evidencia, impacto y cambio realizado.

13 Etapa 8 Representar y documentar la arquitectura

Diagrama de clases

El diagrama debe mostrar las clases relevantes para el proceso elegido, sus responsabilidades y relaciones. Diferencien visualmente dominio, aplicación e infraestructura. Incluyan interfaces de Repository, Application Services y los elementos del dominio que hayan justificado.

Cisco Confidential

Diagrama de clases DDD, parte 1: modelo de dominio

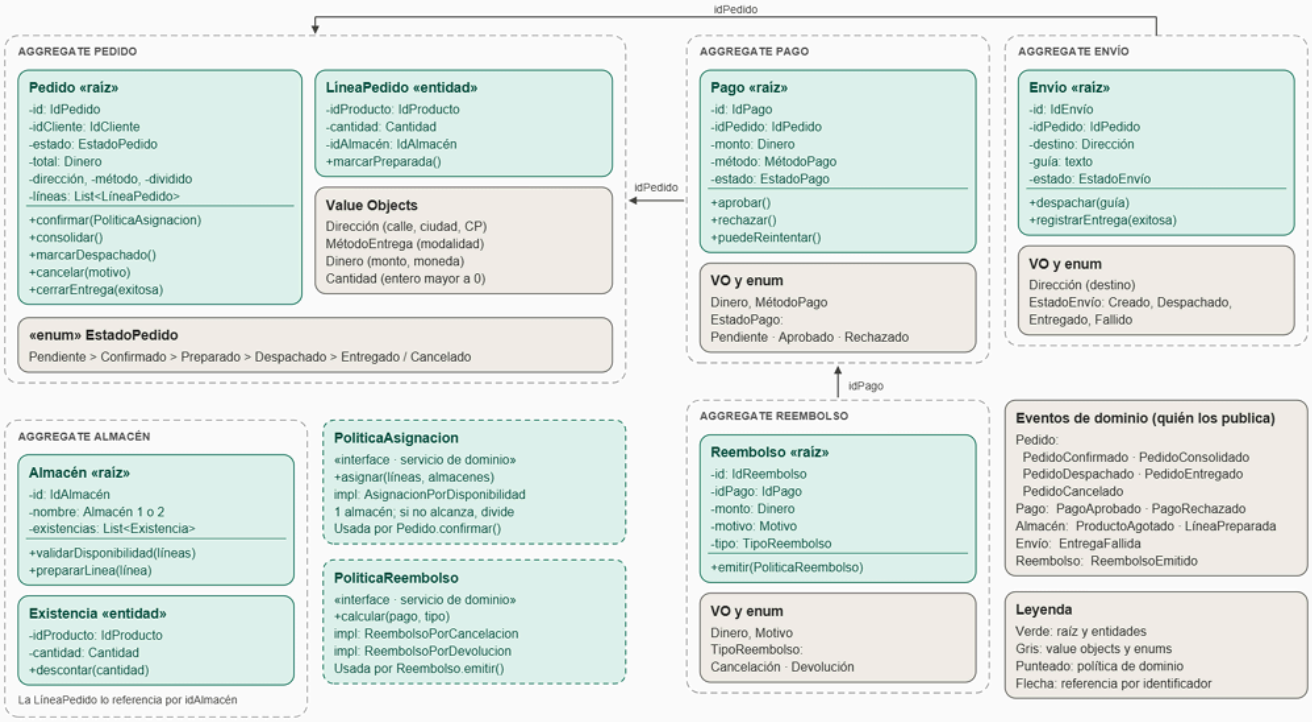
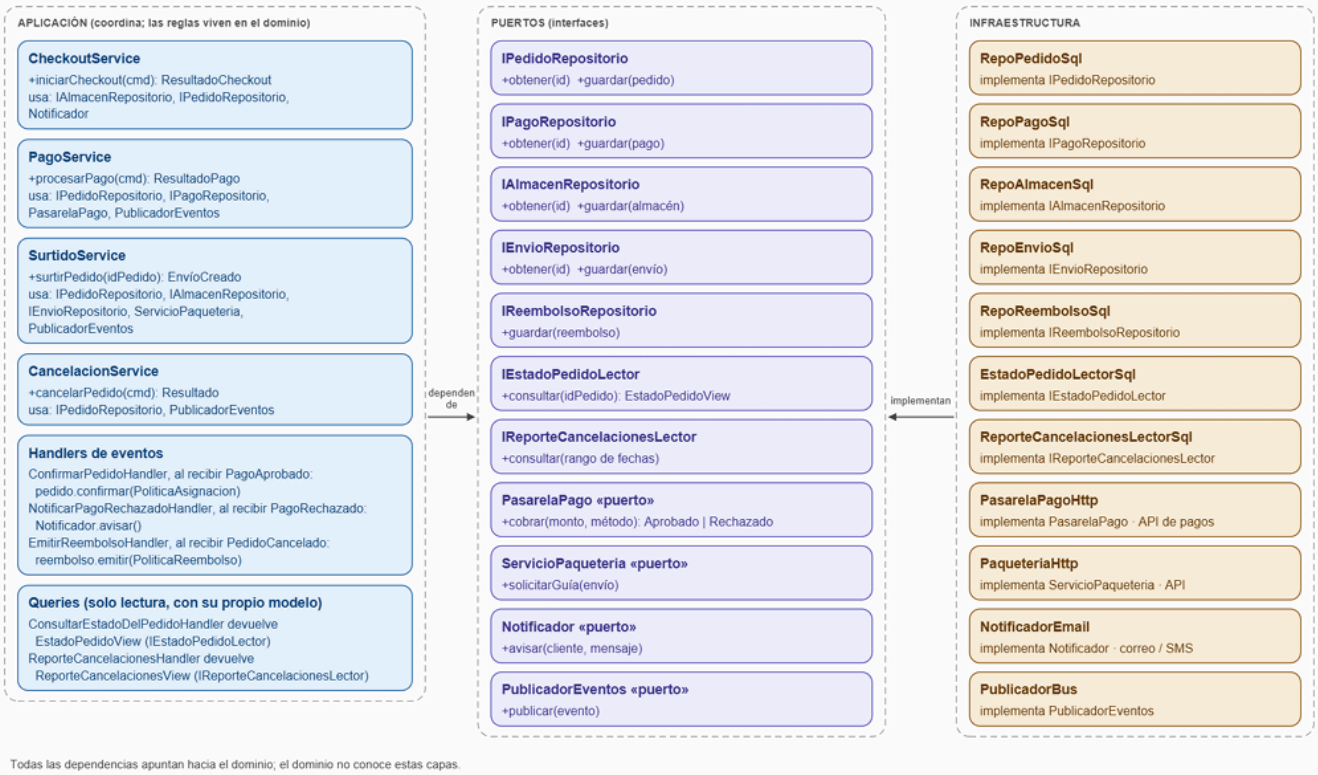


Diagrama de clases DDD, parte 2: aplicación e infraestructura



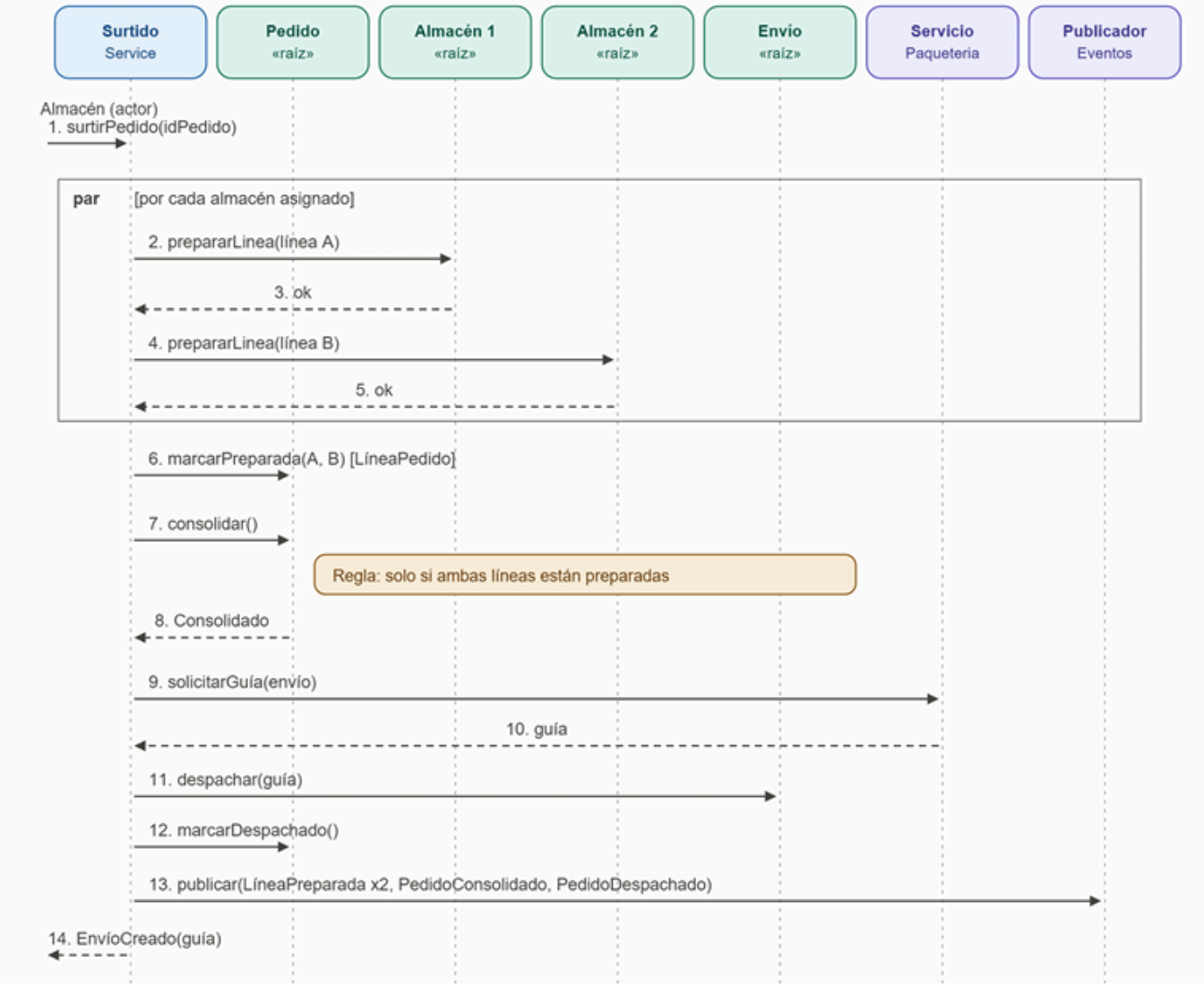
Cisco Confidential

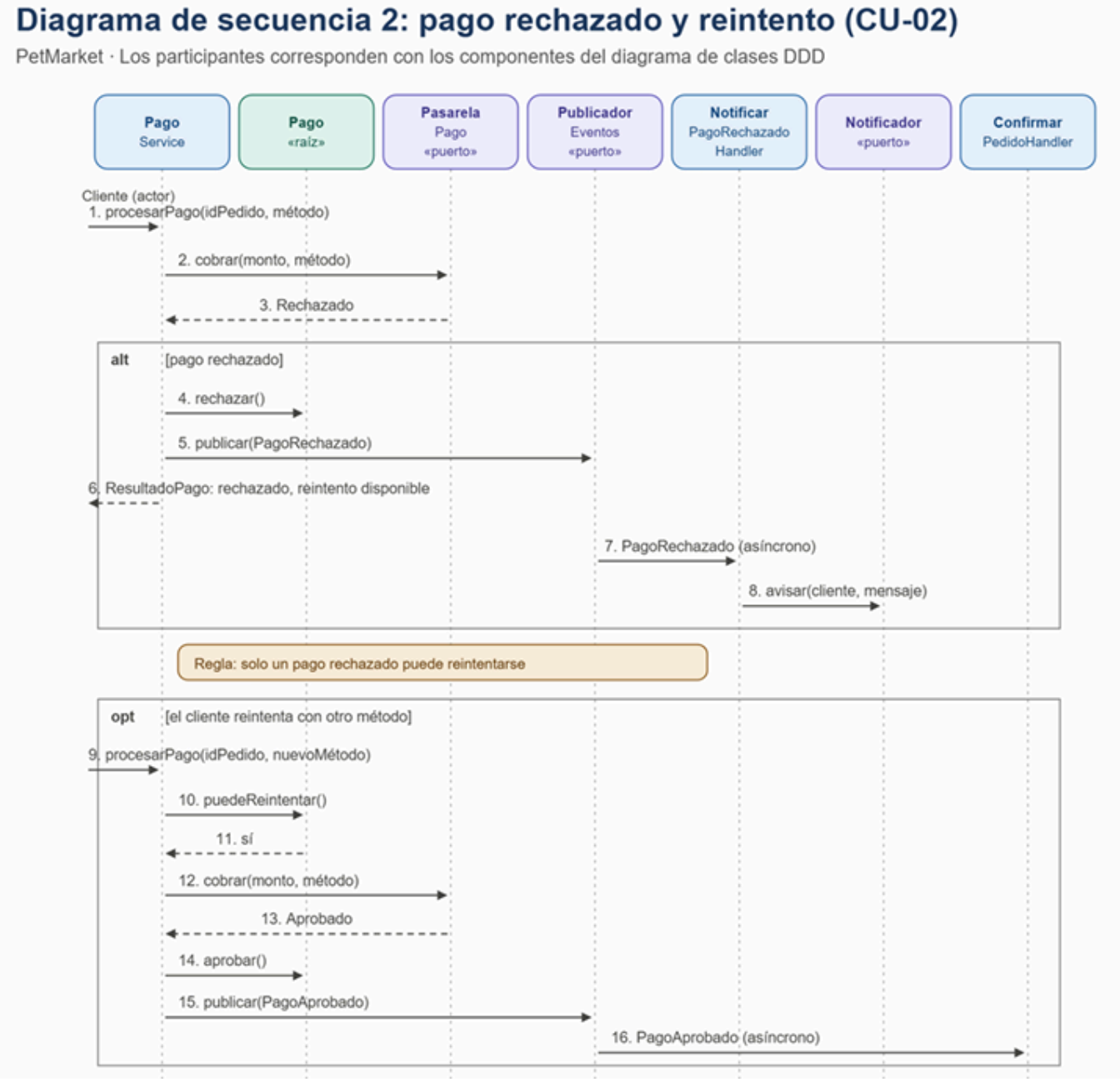
Diagramas de secuencia

Construyan dos diagramas de secuencia. Uno debe representar un flujo principal y el otro un flujo con una regla, excepción o alternativa importante. Los participantes deben corresponder con los componentes del diagrama de clases.

Cisco Confidential

Diagrama de secuencia 1: flujo principal (CU-03, surtir pedido dividido)





Architecture Decision Records

Elaboren al menos tres ADR. Cada ADR documentará una decisión que tenga consecuencias para el diseño. Como mínimo, registren la selección del patrón de lógica de negocio y dos decisiones adicionales tomadas durante la actividad.

Campo del ADR	Contenido que debe redactar el equipo
Titulo	Nombre breve de la decision.
Estado	Propuesta, aceptada, reemplazada o descartada.
Contexto	Problema, restricciones y evidencia relevante de U2.

Cisco Confidential

Campo del ADR	Contenido que debe redactar el equipo
Alternativas	Opciones consideradas y sus consecuencias.
Decision	Opcion elegida y razon concreta.
Consecuencias	Beneficios, costos, riesgos y trabajo posterior.
Trazabilidad	IDs de necesidades, reglas, historias o escenarios relacionados.

ADR-01	
Título	Selección del patrón de lógica de negocio Domain Model
Estado	Aceptada
Contexto	PetMarket requiere administrar un proceso de pedidos que involucra distintas reglas de negocio y la interacción entre Pedido, Línea de pedido, Almacén, Inventario, Pago, Envío y Reembolso. Además, existen diferentes estados y condiciones que deben respetarse, como confirmar un pedido únicamente cuando el pago ha sido aprobado, consolidar los pedidos divididos y evitar cancelaciones después del envío. Dentro del caso se identificaron 14 reglas de negocio (RN-01 a RN-14), así como cuatro casos de uso principales (CU-01 a CU-04). Esta complejidad hace necesario centralizar las reglas y proteger la consistencia del proceso.
Alternativas	• **Transaction Script:** implementa cada caso de uso mediante procedimientos independientes. Es sencillo al inicio, pero puede duplicar reglas y dificultar el mantenimiento. • **Table Module:** organiza la lógica por tablas o grupos de datos. Facilita ciertas operaciones, pero las reglas que involucran varios conceptos pueden quedar dispersas. • **Domain Model:** representa lo conceptos del negocio mediante objetos que contienen sus datos, comportamientos y reglas. Requiere mayor inversión inicial, pero facilita el control de estados y la evolución del sistema.
Decisión	Se selecciona Domain Model como patrón principal, aplicado al núcleo del proceso de pedidos. Las entidades y agregados del dominio serán responsables de mantener sus propias reglas e invariantes. Se utilizará Transaction Script únicamente para operaciones sencillas que no requieran reglas de negocio complejas, como algunas notificaciones y consultas simples.
Consecuencias	• *Beneficios:* centralización de reglas, mayor consistencia de los pedidos y facilidad para incorporar nuevos almacenes o políticas de negocio. • *Costos:* mayor tiempo de diseño, necesidad de definir correctamente las entidades y agregados y trabajo adicional para mapear el dominio a la base de datos. • *Riesgos:* un diseño demasiado complejo si se aplica el patrón a operaciones sencillas. • *Trabajo posterior:* diseñar los agregados Pedido, Almacén, Pago, Envío y Reembolso; definir sus responsabilidades, invariantes y eventos; y comprobar su consistencia con los diagramas.

Cisco Confidential

ADR-01	
Trazabilidad	• Casos de uso: CU-01, CU-02, CU-03 y CU-04. • Reglas de negocio: RN-01 a RN-14. • BPMN del proceso de pedidos, glosario del dominio y reglas relacionadas con inventario, pago, pedidos divididos y cancelaciones.

ADR-02	
Título	Implementación de una Service Layer para coordinar los casos de uso.
Estado	Propuesta
Contexto	PetMarket necesita ejecutar operaciones como iniciar el checkout, procesar pagos, preparar pedidos y registrar cancelaciones. Estas operaciones requieren coordinar varios componentes, incluyendo repositorios, inventario, pagos y objetos del dominio. Si toda la lógica se concentra en los servicios de aplicación, las reglas podrían duplicarse y sería más difícil mantener una separación clara entre la coordinación del proceso y las decisiones propias del negocio.
Alternativas	• **Concentrar toda la lógica en los servicios:** simplifica inicialmente la coordinación, pero genera servicios con demasiadas responsabilidades. • **Distribuir la lógica entre controladores y repositorios:** puede mezclar la comunicación con el usuario, las reglas de negocio y la persistencia. • **Utilizar Service Layer:** los servicios coordinan las operaciones y delegan las reglas de negocio a los objetos del dominio. Esto requiere definir responsabilidades e interfaces, pero mejora la organización.
Decisión	Se propone implementar una Service Layer con servicios de aplicación específicos para cada caso de uso: • *CheckoutService*: coordina la validación del carrito y la disponibilidad del inventario. • *PagoService*: coordina el procesamiento del pago y comunica su resultado. • *PreparacionPedidoService*: coordina la preparación de líneas, la consolidación y la actualización del pedido. • *CancelacionService*: coordina la cancelación y el proceso de reembolso correspondiente.
Consecuencias	• *Beneficios:* responsabilidades mejor definidas, menor duplicación de reglas y mayor facilidad para probar cada caso de uso. • *Costos:* creación de servicios, interfaces y métodos específicos para las operaciones del negocio. • *Riesgos:* que los servicios terminan concentrando reglas que deberían pertenecer al dominio. • *Trabajo posterior:* definir las entradas y salidas de cada servicio, sus dependencias, los límites transaccionales y el manejo de errores.

Cisco Confidential

ADR-02	
Trazabilidad	• Casos de uso: CU-01, CU-02, CU-03 y CU-04. • Reglas de negocio: RN-01 a RN-14. • Componentes relacionados: *CheckoutService, PagoService, PreparacionPedidoService, CancelacionService, PedidoRepository* e *InventarioAlmacenRepository.* • Flujo BPMN del pedido y sus excepciones de pago, inventario, preparación y cancelación.

ADR-03	
Título	Separación de Commands y Queries mediante CQRS básico.
Estado	Propuesta
Contexto	PetMarket necesita realizar operaciones que modifican el estado de los pedidos, como iniciar un checkout, procesar un pago, preparar una línea o cancelar una compra. También requiere consultar información sin modificar el estado del sistema, como el estado de un pedido o los reportes de cancelaciones y devoluciones. Si ambas responsabilidades se mezclan, las consultas podrían quedar acopladas a la lógica de modificación y resultar más difíciles de mantener. Por ello, se considera conveniente distinguir las operaciones que modifican datos de aquellas que únicamente recuperan información.
Alternativas	• **Utilizar los mismos métodos para escritura y consulta:** reduce la cantidad inicial de componentes, pero mezcla responsabilidades. • **Separar Commands y Queries con CQRS básico:** permite definir contratos, manejadores y resultados específicos para cada tipo de operación, aunque requiere componentes adicionales. • **Implementar CQRS con bases de datos separadas:** ofrece una separación mayor, pero incrementa la complejidad de infraestructura y sincronización**.**
Decisión	Se propone implementar CQRS básico sin separar las bases de datos. Los Commands modificarán el estado del sistema, mientras que las Queries devolverán información sin realizar cambios. Se contemplan los siguientes ejemplos: <u>Commands</u> • *IniciarCheckoutCommand:* inicia el checkout y permite crear el pedido. • *ProcesarPagoCommand:* procesa el pago y registra su resultado. • *CancelarPedidoCommand:* solicita la cancelación y coordina el reembolso cuando corresponda. • *MarcarLineaPreparadaCommand:* registra la preparación de una línea del pedido. <u>Queries</u> • *ConsultarEstadoDelPedidoQuery:* obtiene el estado actual del pedido. • *ConsultarCancelacionesYDevolucionesQuery:* genera un reporte a partir de un rango de fechas.

Cisco Confidential

ADR-03	
Consecuencias	• *Beneficios:* separación clara de responsabilidades, consultas más fáciles de adaptar a las necesidades de lectura y mayor claridad en las operaciones que modifican el estado. • *Costos:* creación de Commands, Queries, Handlers y modelos de lectura específicos. • *Riesgos:* duplicación innecesaria de estructuras si la separación se aplica a operaciones demasiado simples. • *Trabajo posterior:* definir los contratos de entrada y salida, implementar los Handlers y verificar que las consultas no modifiquen información del dominio.
Trazabilidad	• Casos de uso: CU-01, CU-02, CU-03 y CU-04. • Reglas de negocio: RN-01 a RN-14, según la operación ejecutada. • Operaciones relacionadas: *IniciarCheckoutCommand, ProcesarPagoCommand, CancelarPedidoCommand, MarcarLineaPreparadaCommand, ConsultarEstadoDelPedidoQuery* y *ConsultarCancelacionesYDevolucionesQuery*. • Flujo del proceso de pedidos y necesidades de seguimiento del estado y consulta de cancelaciones y devoluciones.

Matriz de Trazabilidad

1. Casos de uso → reglas → componentes

CU	Reglas	Historias / BPMN	Componentes de dominio	Aplicación (servicio / Command / Query)	Diagrama	ADR
CU-01 Iniciar checkout y validar inventario	**RN-01, RN-02, RN-03**	**US02-US04 (por confirmar); gateway de stock**	**Aggregate Almacén, Existencia, validarDisponibilidad(), evento ProductoAgotado, Pedido (estado Pendiente)**	**CheckoutService.iniciarCheckout / IniciarCheckoutCommand / IniciarCheckoutHandler**	**Clases (partes 1 y 2)**	**ADR-01, 02, 03**
CU-02 Procesar pago y confirmar pedido	**RN-04, RN-05, RN-06**	**US05-US08 (por confirmar); gateway ¿Pago aprobado?**	**Aggregate Pago (aprobar, rechazar, puedeReintentar), Pedido.confirmar(), PagoAprobado, PagoRechazado, PedidoConfirmado**	**PagoService.procesarPago / ProcesarPagoCommand / ProcesarPagoHandler; ConfirmarPedidoHandler, NotificarPagoRechazadoHandler**	**Clases; Secuencia 2**	**ADR-01, 02, 03**
CU-03 Preparar, empacar y despachar pedido	**RN-07 a RN-11**	**US09, US10, US11, US12, US14; gateway ¿Requiere 2 almacenes?**	**PoliticaAsignacion, LineaPedido.marcarPreparada, Almacén.prepararLinea, Pedido.consolidar / marcarDespachado / cerrarEntrega, Aggregate Envío, eventos LíneaPreparada, PedidoConsolidado, PedidoDespachado, PedidoEntregado, EntregaFallida**	**SurtidoService.surtirPedido (en la Etapa 4: PreparacionPedidoService) / MarcarLineaPreparadaCommand / MarcarLineaPreparadaHandler**	**Clases; Secuencia 1**	**ADR-01, 02, 03**

Cisco Confidential

CU-04 Cancelar pedido antes del envío	RN-12, RN-13, RN-14	US15, US16, US17; gateway ¿Cancela antes del envío?	Pedido.cancelar(motivo), PedidoCancelado, Aggregate Reembolso (emitir), PoliticaReembolso, ReembolsoEmitido	CancelacionService.cancelarPedido / CancelarPedidoCommand / CancelarPedidoHandler; EmitirReembolsoHandler	Clases (sin secuencia propia)	ADR-01, 02, 03
Consulta de estado del pedido	Ninguna (solo lectura)	US13	No pasa por el dominio	ConsultarEstadoDelPedidoQuery → EstadoPedidoView (IEstadoPedidoLector)	Clases parte 2	ADR-03
Reporte de cancelaciones y devoluciones	Ninguna (solo lectura)	US18	No pasa por el dominio	ConsultarCancelacionesYDevolucionesQuery → ReporteCancelacionesView (IReporteCancelacionesLector)	Clases parte 2	ADR-03

2. Reglas de negocio → dónde se protegen

Regla	Descripción	Dónde vive	Evidencia visual	CU
RN-01	El stock debe estar disponible en el almacén	Almacén.validarDisponibilidad(); invariante: la cantidad nunca es negativa	Clases p.1	CU-01
RN-02	Sin stock: notificar “producto agotado”	Evento ProductoAgotado + Notificador; ResultadoCheckout con lista de agotados	Clases p.2	CU-01
RN-03	Solo se avanza si el stock es suficiente	CheckoutService (coordina); el Pedido solo se crea si el stock está confirmado	Clases p.2	CU-01
RN-04	Dirección y método de entrega válidos antes del pago	Value Objects Dirección y MétodoEntrega en Pedido	Clases p.1	CU-02
RN-05	El pedido se confirma solo con pago aprobado	Pedido.confirmar(), disparado por PagoAprobado	Secuencia 2 (paso 16)	CU-02
RN-06	Pago rechazado: notificar y reintentar	Pago.rechazar(), puedeReintentar() + NotificarPagoRechazadoHandler	Secuencia 2 (pasos 4-14)	CU-02
RN-07	Pedido con 2 almacenes: cada uno prepara su línea	PoliticaAsignacion, LineaPedido.idAlmacén, Almacén.prepararLinea()	Secuencia 1 (bloque par)	CU-03
RN-08	Un solo almacén prepara el pedido completo	PoliticaAsignacion (asigna a 1 almacén si alcanza)	Clases p.1	CU-03
RN-09	Empacar y consolidar antes del despacho	Pedido.consolidar() (solo si todas las líneas están preparadas)	Secuencia 1 (pasos 6-8)	CU-03
RN-10	Entrega exitosa: “pedido entregado”	Envío.registrarEntrega(true), Pedido.cerrarEntrega(), PedidoEntregado	Clases p.1	CU-03
RN-11	Entrega fallida: nuevo pedido y notificación	Evento EntregaFallida (estado Fallido en Envío)	Clases p.1 (solo el evento)	CU-03
RN-12	Cancelar solo antes del envío	Pedido.cancelar() rechaza si el estado es Despachado	Clases p.1	CU-04
RN-13	Registrar cancelación y notificar al almacén	Evento PedidoCancelado	Clases p.1	CU-04
RN-14	La cancelación genera reembolso	Reembolso.emitir(PoliticaReembolso) vía EmitirReembolsoHandler	Clases p.2	CU-04

Cisco Confidential

3. ADR → qué justifican

ADR	Decisión	Estado	CU	Reglas	Componentes
ADR-01	Domain Model (Transaction Script solo para operaciones simples)	Aceptada	CU-01 a 04	RN-01 a RN-14	Aggregates Pedido, Almacén, Pago, Envío, Reembolso
ADR-02	Service Layer	Propuesta	CU-01 a 04	RN-01 a RN-14	CheckoutService, PagoService, PreparacionPedidoService / SurtidoService, CancelacionService, repositorios
ADR-03	CQRS básico	Propuesta	CU-01 a 04	RN-01 a RN-14 según operación	4 Commands, 2 Queries, Handlers, Views

Cisco Confidential

4. Huecos e inconsistencias a corregir antes de entregar

1. Numeración de reglas inconsistente. La Etapa 4 cita RN-05/06 para preparación y RN-07/08 para cancelación, y la Etapa 2 dice “no se cancela después del envío (RN-11)”. Según la Etapa 1 son RN-09, RN-12 y RN-13. ELEGIR_PATRONES y las respuestas de defensa usan RN-04 y RN-07 con otro significado. Unificar con los IDs de la Etapa 1.
2. Numeración de ADR inconsistente. El PDF de ADR usa ADR-01 a 03, pero la Etapa 9 y ELEGIR_PATRONES hablan de ADR-001 a ADR-004. Falta un ADR de Unit of Work y pasarela de pago fuera de la transacción.
3. Nombre del servicio de preparación: PreparacionPedidoService (Etapa 4 y ADR-02) frente a SurtidoService (diagrama de clases). Elegir uno.
4. RN-11 (entrega fallida) no tiene flujo: solo existe el evento EntregaFallida. No hay servicio ni secuencia para generar el pedido nuevo. Esto se relaciona con la falta de EnvioService y de CU-05.
5. RN-13 (notificar al almacén) no tiene handler: en el diagrama, PedidoCancelado solo dispara el reembolso.
6. CU-04 no tiene secuencia propia y su “Excepción” quedó vacía en la Etapa 1.
7. Las secuencias cubren solo CU-03 (flujo normal) y CU-02 (excepción). CU-01 y CU-04 no se recorren en ningún diagrama.
8. Secuencia 2: el paso 16 publica PagoAprobado, pero ConfirmarPedidoHandler no muestra la llamada a confirmar().
9. Historias de usuario: los rangos US02-US04 (CU-01) y US05-US08 (CU-02) son supuestos; verificarlos contra el BRD.
10. Modelado: ELEGIR_PATRONES mostraba Pago dentro de Pedido y Carrito como Aggregate; en el diagrama final Pago es Aggregate separado y Carrito quedó fuera. La matriz sigue el diagrama final.

Herramientas permitidas

Pueden utilizar draw.io, PlantUML, Mermaid u otra herramienta que permita entregar diagramas legibles. Los archivos fuente deben incluirse en el repositorio; no entreguen únicamente capturas de pantalla.

14 Vibe Coding con IA

Usarán IA para convertir descripciones del negocio en alternativas de arquitectura, generar estructura inicial, revisar código y producir borradores de diagramas. El equipo sigue siendo responsable de la arquitectura y debe verificar cada resultado contra los documentos de la Unidad 2.

Cisco Confidential

Ciclo de trabajo esperado

Paso	Accion	Evidencia
1	Preparar contexto	Seleccionar fragmentos relevantes del BRD, historias, reglas, BPMN y glosario.
2	Solicitar propuesta	Pedir una alternativa concreta y exigir que la IA declare supuestos.
3	Comparar	Contrastar la respuesta con el caso y con los conceptos de la unidad.
4	Corregir	Modificar nombres, responsabilidades, limites y dependencias que no correspondan.
5	Validar	Revisar trazabilidad, consistencia entre diagramas y comportamiento esperado.
6	Registrar	Guardar prompt, respuesta, decision humana y version resultante.

Prompts de apoyo

Los siguientes formatos son puntos de partida. Completen los corchetes con información de su empresa y eviten enviar datos que no formen parte del caso académico.

- Patrones: Con base en [reglas, cambios y casos de uso], compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.
- Dominio: Revisa este glosario y estas reglas [contenido]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.
- Service Layer: A partir de estos casos de uso [contenido], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.
- SOLID: Analiza este diseño [contenido] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.
- Diagramas: Convierte esta arquitectura validada [contenido] en un borrador PlantUML o Mermaid. No agregues clases, reglas o relaciones que no aparezcan en la descripción.

Bitacora obligatoria de IA

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1	Entender la actividad y revisar la Unidad 2	Se subieron el PDF de PetMarket y la guía de la Actividad 3, y se pidió ayuda con el componente de lógica de negocio	Lista de hallazgos: orden de “Confirmar pedido” y “Procesar pago”, gateway de stock, Gherkin y matriz ausentes, reglas sin ID, portada con otro curso	Los IDs RN-01 a RN-08 los propuso la IA; no existen en U2	(Verificar los hallazgos contra el PDF y decidir cuáles registrar)

Cisco Confidential

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
2	Entender la actividad y revisar la Unidad 2	Se pidió empezar por la tabla de casos de uso y la comparación de patrones	CU-01 a CU-04 con trazabilidad a historias; tabla comparativa que favorece Domain Model	El argumento de “cambios esperados” es supuesto, porque el BRD no define metas; la IA inclinó la decisión hacia Domain Model	(Aceptar, ajustar o cambiar el patrón, y redactar ADR-001)
3	Service Layer, Repository y Unit of Work (Etapas 4 y 5)	Se pidió desarrollar ambas etapas	Cuatro servicios, tres repositorios, Unit of Work con eventos tras el commit	Supuestos: pasarela de pago fuera de la transacción; nombres provisionales de los objetos del dominio; el BRD no dice si el inventario se reserva	(Decidir sobre la pasarela y validar la reserva de inventario con el stakeholder)
4	Mapa de dominio y CQRS (Etapas 3 y 6)	Se pidió desarrollar ambas etapas	Aggregates, Value Objects, eventos, diagrama Mermaid, fichas, Commands y Queries	Supuestos: estados del Pedido, `Pago` dentro de `Pedido`, una línea con un solo almacén, `Cliente` y `Reembolso` descartados	(Confirmar o corregir cada supuesto)

15 Etapa 9 Validar la propuesta

Antes de entregar, recorran cada caso de uso sobre los diagramas y el diseño. El equipo debe poder explicar qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persisten los cambios y qué resultado se devuelve.

Cisco Confidential

Validacion	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	**Parcial.** CU-01 a CU-04 se relacionan con US02 a US11, US15 y US16, y cada servicio del diagrama de clases responde a un caso de uso. Excepciones: `EnvioService` y `EnvioRepository` (US14) no tienen caso de uso, porque no hay CU-05, y US17 (Atención al Cliente) queda fuera del diseño detallado. Hay que agregar CU-05 o justificarlos como corrección de alcance, y falta la matriz de trazabilidad formal.
Lenguaje	¿Los mismos terminos significan lo mismo en BRD, UML, codigo y ADR?	**Parcial.** Los nombres de clases, servicios y eventos coinciden entre el mapa de dominio, el diagrama de clases, las secuencias y los ADR. Pendiente: agregar al glosario `Despachado`, `Reintento de pago`, `Guía de envío`, `Aggregate` y `Domain Event`, y decidir si los identificadores del código llevan acento (`Envio`, `Almacen`) o no, y aclararlo en el glosario para que no parezcan términos distintos.

Cisco Confidential

Validacion	Pregunta	Resultado del equipo
Responsabilidad	¿Las reglas permanecen en el dominio o en el patron elegido con una justificacion?	**Cumple, con dos puntos a defender.** Las reglas de cancelación (RN-07), reintento (RN-04) y consolidación (RN-06) viven en `Pedido`, y la asignación de almacén en `AsignadorDeAlmacenes`. Los servicios solo coordinan: cargar, invocar, guardar y llamar a la pasarela. Puntos dudosos: `Pago` dentro de `Pedido` y los tres métodos de `PreparacionPedidoService`.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	**Parcial.** Los participantes y mensajes de las dos secuencias existen en el diagrama de clases (`PagoService`, `CancelacionService`, `Pedido`, `PedidoRepository`, `UnitOfWork`, `PasarelaPago` y los métodos usados). Discrepancia detectada: la secuencia 1 no llama a `confirmar()` después de `registrarPagoAprobado()`, aunque `Pedido` tiene ese método. Hay que decidir si `registrarPagoAprobado()` confirma internamente o si se agrega la llamada.

Cisco Confidential

Validacion	Pregunta	Resultado del equipo
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	**Cumple en el diseño.** El dominio solo conoce las interfaces de Repository; SQL, el sistema de inventario y la pasarela están en adaptadores de infraestructura. Aún no está verificado en código, porque el código base no existe todavía.
Transaccion	¿El limite de la operacion y su manejo de error estan definidos?	**Parcial.** Está definido dónde empieza y termina el Unit of Work (ADR-002 y ADR-003) y qué pasa ante un error de dominio (`rollback`). Siguen abiertos: idempotencia del cobro, falla del reembolso después del `commit`, edición simultánea del mismo pedido y el máximo de reintentos de pago.
IA	¿Cada aporte de IA fue revisado y su uso esta registrado?	**Parcial.** La bitácora tiene 5 iteraciones con objetivo, respuesta útil y supuestos. Faltan las iteraciones de los diagramas (clases, secuencias, archivos draw.io) y, sobre todo, **la decisión humana de cada una**, que es lo que evalúa la rúbrica.

16 Entregables

Entregable	Contenido minimo
01 Informe de arquitectura	Documento PDF con portada, alcance, comparación de patrones, mapa de dominio, Service Layer, Repository, Unit of Work, CQRS, revisión SOLID, ADR y conclusiones.

Cisco Confidential

Entregable	Contenido minimo
02 Diagrama de dominio y clases	Archivo fuente y exportación legible en PDF o PNG.
03 Diagramas de secuencia	Dos archivos fuente y sus exportaciones.
04 Matriz de trazabilidad	Relación entre necesidades, reglas, historias, casos de uso, componentes y decisiones.
05 Bitacora de IA	Prompts, respuestas relevantes, errores detectados, correcciones y decisiones humanas.
06 Codigo base	Estructura de carpetas y código mínimo que represente la arquitectura propuesta.
07 Repositorio GitHub	Todos los archivos organizados, historial de commits y README con instrucciones de revisión.
08 Presentacion y defensa	Exposición breve donde todos los integrantes puedan explicar y defender decisiones.

Estructura sugerida del repositorio

- docs Informe, ADR, trazabilidad y bitacora de IA
- diagrams Archivos fuente y exportaciones
- src Dominio, aplicacion e infraestructura
- tests Escenarios o pruebas seleccionadas
- README Empresa, alcance, instrucciones y enlaces a entregables

17 Rubrica de evaluacion

Criterio	Evidencia esperada	Puntos
Continuidad y trazabilidad	Usa los productos de U2, conserva el alcance y relaciona decisiones con evidencia.	15
Seleccion del patron	Compara alternativas y justifica la eleccion con criterios concretos del caso.	15
Mapa de dominio	Distingue responsabilidades, limites, lenguaje, invariantes y eventos de forma coherente.	20
Service Layer y persistencia	Coordina casos de uso, separa reglas y define Repository y Unit of Work.	15
CQRS y SOLID	Separa operaciones con criterio y revisa el diseño mediante evidencia concreta.	10
UML y ADR	Los diagramas son consistentes y las decisiones registran contexto, alternativas y consecuencias.	15
Uso responsable de IA	Presenta iteraciones, detecta errores y explica las decisiones humanas.	5
Repositorio y defensa	Entrega organizada, legible y defendida por todos los integrantes.	5

Total 100 puntos

Cisco Confidential

18 Criterios de integridad academica

- La IA puede proponer, revisar y explicar, pero no reemplaza la justificación del equipo.
- Toda afirmación sobre el negocio debe provenir del caso o identificarse como supuesto pendiente de validación.
- El equipo debe conservar la autoría de sus decisiones y poder explicarlas sin leer una respuesta generada.
- El código, los diagramas o los textos generados deben revisarse antes de integrarlos al repositorio.
- Copiar la arquitectura de otro equipo o entregar contenido que no corresponde a la empresa asignada se considera incumplimiento.

19 Lista final de comprobacion

- [ ] Trabajamos con la misma empresa y el mismo alcance de la Unidad 2.
- [ ] La selección del patrón está respaldada por evidencia del caso.
- [ ] El mapa de dominio usa el lenguaje ubicuo del proyecto.
- [ ] Cada Application Service representa un caso de uso y coordina sin concentrar todas las reglas.
- [ ] Las interfaces de Repository hablan en términos del dominio.
- [ ] Los Commands y Queries tienen responsabilidades distintas y justificadas.
- [ ] Revisamos el diseño con SOLID y corregimos los hallazgos relevantes.
- [ ] Los diagramas de clase y secuencia son consistentes.
- [ ] Incluimos al menos tres ADR y una matriz de trazabilidad actualizada.
- [ ] La bitácora demuestra cómo usamos y validamos la IA.
- [ ] El repositorio contiene archivos fuente, exportaciones y un README claro.
- [ ] Todos los integrantes pueden defender las decisiones tomadas.

20 Preguntas para la defensa

- ¿Qué evidencia del negocio influyó más en la selección del patrón?

R=El BPMN tiene seis carriles y tres finales, así que el pedido interactúa con pago, almacenes, paquetería y atención al cliente.

El BRD pide reglas explícitas para las excepciones (pago rechazado, producto agotado, pedido dividido, cancelación, entrega fallida).

Las historias US09 a US11 (pedido dividido por línea), US15 y US16 (cancelar solo antes de “despachado” y reembolsar).

Aclaren con honestidad que “cuántas veces cambiarán las reglas” no está en la Unidad 2, así que ese argumento es un supuesto y no el que sostiene la decisión.

- ¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?

R=

Candidatas de lo que trabajamos:

- La asignación de almacén a cada línea: usa existencias de varios almacenes, así que no cabe en una sola entidad. Terminó como Domain Service (`AsignadorDeAlmacenes`).
- El cobro a la pasarela: ¿dentro o fuera de la transacción? Quedó fuera (ADR-002).
- `Pago` dentro de `Pedido` o aparte: sigue como decisión dudosa.

Cisco Confidential

- ¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?

R=

- La regla más clara es RN-07: no se cancela un pedido ya `Despachado`, y vive en `Pedido.cancelar()`.
- Si estuviera en `CancelacionService`, otro servicio (por ejemplo, uno de atención al cliente o un proceso nocturno) podría llamar a `pedido.cancelar()` sin pasar por esa validación y dejar el pedido en un estado inválido. Es la diferencia entre proteger la regla en el dominio y recordar aplicarla en cada servicio.
- Con RN-04 (límite de reintentos) pasa lo mismo.

- ¿Qué cambió después de la revisión con SOLID?

R=Todavía no la hemos hecho, así que no hay nada que contar. Hagan la Etapa 7 antes de la defensa, con al menos cinco hallazgos y el cambio realizado en cada uno. Esta pregunta se responde con esos cambios, y si dicen que no cambió nada, suena a que no se revisó.

- ¿Qué propuesta de la IA rechazaron o corrigieron y por qué?

R=Esto solo lo pueden responder ustedes, y debe coincidir con la columna “Decisión del equipo” de la bitácora. Para ayudarles a recordar, estos son puntos donde mis propuestas tuvieron problemas o supuestos que ustedes deberían haber revisado:

- Los IDs de reglas RN-01 a RN-08 los propuse yo, no existen en el BRD.
- Servicios y Handlers duplicados, que hubo que unificar.
- `EnvioService` que faltaba en la primera versión.
- La flecha “usa” hacia Value Objects que se ve mal en el diagrama de clases.
- La secuencia de pago que no llama a `confirmar()`.

- ¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?

R=

- Con Domain Model aceptan más estructura y diseño inicial, más clases y más disciplina para no escribir reglas en los servicios.
- Si el negocio confirmara que las reglas casi no cambian, Transaction Script habría sido más barato.
- Una consecuencia relacionada: el modelo de lectura puede mostrar un estado desactualizado un momento (ADR-004), lo que choca con el objetivo del BRD de información actualizada.

Cisco Confidential

https://github.com/emilianosanjua/DISENO_ARQUITECTURAL_DEL_SISTEMA